



Urban Intelligence & Transit ADAS

Technical Architecture, AI Pipelines & Technology Stack Report

Project ID: SIH26124 (Urban Innovators)

Version: 2.4 Production Baseline

Deployment: Transit Fleets (UPSRTC / BMTC / DTC)

Architecture: 100% Offline Edge-First Computing

ZERO EXTERNAL API COST

LOCAL EDGE CV & AI

SPATIAL-TEMPORAL CLUSTERING

MULTI-HAZARD ADAS

1. Executive Summary & Architecture Overview

The **Urban Intelligence & Transit ADAS Platform** is an edge-deployed computer vision and geospatial telematics system designed to turn standard public transit buses into continuous mobile city scanners. Video feeds from dashboard cameras and synchronized GPS coordinates are analyzed onboard in real-time or batch execution, generating structured municipal maintenance registers and proactive Driver Assistance (ADAS) warnings without requiring cloud connectivity, heavy external API subscriptions, or expensive dedicated survey vehicles.

HARDWARE OPTIMIZATION

NVIDIA Jetson Orin / Nano / Onboard Fleet PC

INFERENCE ENGINE

Local YOLOv8 Nano + OpenCV Optics (Zero Cloud Dependency)

GEOSPATIAL MAPPING

Hardware-Accelerated 3D WebGL (Deck.gl / PyDeck)

INCIDENT CLUSTERING

85-Meter Spatial-Temporal Radius Haversine Engine

2. Frontend Technology Stack (Client-Side Command UI)

The frontend is built as an **Executive Urban Intelligence Command Center** featuring an ultra-premium dark glassmorphic design system tailored for municipal commissioners, PWD engineers, and traffic authorities.

Component	Technology Used	Implementation Details & Functional Impact
Core Framework	Streamlit 1.35.0+	Reactive component-driven architecture, persistent sidebar navigation, interactive session state, and native high-speed reactive UI rendering.
Design & Styling	Embedded CSS3	Dark glassmorphism (backdrop-filter: blur(16px)), custom gradients, cyber-cyan neon telemetry accents, and responsive layout containers.
Color Palette	Curated Cyber Slate	Background (#070a13), Glass Cards (#0f172a), Accents (#38bdf8 Cyan, #ef4444 Danger, #10b981 Normal).
Typography	Google Web Fonts	Plus Jakarta Sans for executive typography and navigation; JetBrains Mono for GPS telemetry, speed readouts, and timestamps.
3D GIS Mapping	PyDeck (Deck.gl 0.8+)	GPU-accelerated WebGL layers (ScatterplotLayer, PathLayer, HexagonLayer) over free Carto vector tiles — zero Mapbox or Google Maps key required.
Analytics & Charts	Pandas / Streamlit Native	Traffic congestion density bar charts, multi-hazard severity matrices, and hourly incident frequency distributions.
Evidence Galleries	Pillow (PIL)	Multi-column visual proof galleries displaying bounding-box annotated frames with embedded telemetry HUD banners.
Data Exporting	CSV UTF-8 Stream	One-click CSV data export buttons for direct work-order integration with PWD, Municipal Jal Sansthan, and Traffic Police departments.

3. Backend & Core Computing Layer

The computational backend is written in modular Python, providing end-to-end telemetry synchronization, AI pipeline scheduling, and inter-process reliability.

Module	Technology Used	Implementation Details & Functional Impact
Core Language	Python 3.10 – 3.14	High-performance pipeline coordination, file I/O streaming, and asynchronous AI model execution.
Data Aggregation	Pandas 2.0+	Telemetry rolling averages, road corridor classification, GPS interpolation, and multi-sensor registry consolidation.
Array Operations	NumPy 1.24+	High-speed image matrix slicing, color-space manipulations, contour area integrals, and coordinate geometry.
Cross-Platform I/O	pathlib.Path	Strictly platform-agnostic file handling ensuring 100% parity across Windows, Ubuntu/Debian Linux, and macOS.
Environment Tooling	tools/fix_portable_venv.py	Custom dynamic virtual environment re-linker that repairs Python binary interpreter links when migrating projects across host laptops.
Server Runtime	.streamlit/config.toml	Production cloud parameters (headless = true, enableCORS = false, 500MB upload capacity).

4. Artificial Intelligence & Computer Vision Pipelines

All computer vision models and algorithmic detectors run locally in the ai_engine/ package, orchestrated by the master pipeline:

1. 🚗 **Vehicle Detection & Traffic Bottleneck Engine (vehicle_detector.py)**

 - **Architecture:** YOLOv8 Nano (yolov8n.pt) + ByteTrack algorithm (bytetrack.yaml).
 - **Classes Tracked:** Cars, Motorcycles/Bikes, Transit Buses, Commercial Trucks.
 - **Bottleneck Heuristics:** Identifies congestion choke points when vehicle density surges while transit speed drops below 36 km/h.
2. 🕳️ **Smart Pothole & Road Cavity Engine (smart_pothole_detector.py)**

 - **Technology:** Morphological Blackhat transform + Surface Depression contouring.
 - **Depth & Area Estimation:** Calculates estimated depth (cm) and depression area (px), rejecting flat asphalt patches and shadows.
 - **Department Attribution:** Generates instant cold-mix asphalt repair tickets routed directly to PWD / NHA.
3. 🌊 **Dual-Channel Waterlogging Optics (waterlogging_detector.py)**

 - **Dual Color-Space Optics:** Dual-channel **LAB** (stagnant dark water pools) + **HSV** (specular water surface glint).
 - **Artifact Rejection:** Sobel horizontal gradient filters suppress painted white road lane stripes and dry asphalt glare.
 - **Action Triggers:** Automatically flags municipal drainage suction pump dispatch tickets when water coverage exceeds critical thresholds.
4. 🚶 **Pedestrian Safety & School Zone ADAS (pedestrian_detector.py)**

 - **Model:** YOLOv8 Person class with spatial proximity heuristics.
 - **ADAS Warning:** Issues priority driver collision avoidance audio/visual alarms when school children or pedestrians enter the vehicle transit path.

5. 🔍 **ANPR & Speed Offender System (anpr_detector.py)**

- **Localization:** Aspect-ratio edge contour bounding box localization.
- **Corridor-Aware Logic:** Geographical state registration prefixes (UP-53 Gorakhpur, KA-04 Bengaluru, DL-01 Delhi, MH-02 Mumbai).
- **Violation Thresholds:** Flags reckless driving and urban over-speeding (>48 km/h or >32 km/h in school zones) for Central Traffic Police e-Challan.

6. 📍 **Spatial-Temporal Clustering & Multi-Hazard Dispatch (incident_generator.py)**

- **Algorithm:** Haversine GPS spherical distance matrix grouping multi-sensor hazards within an **85-meter radius**.
- **Multi-Hazard Escalation:** Co-occurring defects (e.g., Severe Pothole + Waterlogging Pool + Traffic Jam) escalate to **Priority 98-100 CRITICAL**, initiating coordinated inter-departmental emergency response.

5. Data Storage, Telemetry Schema & Cloud Deployment

Component	Format / Location	Operational Role
Master Incident Registry	data/incidents.csv	Unified ledger of all detected urban defects containing GPS, severity, timestamps, and assigned work orders.
Sensor Stream Logs	data/*_results.csv	Granular sensor logs for ANPR, Traffic, Potholes, Waterlogging, Pedestrians, Sanitation, and Signal faults.
ICCC Cloud Payload	edge_metadata_payload.json	REST-compliant telemetry payload ready for direct ingestion by Smart City Integrated Command and Control Centers.
Visual Evidence Cache	evidence/route_*/*.jpg	High-resolution annotated JPG frames with GPS coordinates, Bus ID, and speed HUD banners.
System Binaries	packages.txt	Debian Linux dependencies: ffmpeg (video streaming), libg11 (OpenGL for OpenCV), and libglib2.0-0.
Cloud Hosting	Streamlit Community Cloud	Docker Linux container runtime deployed directly from GitHub (main branch).